Estructuras de Datos Dinámicas

Colas

Fecha de revisión:

Ing. Sofía Pérez

Agenda

1. Cola Lineal
2. Funcionamiento
3. Operaciones más comunes
4. Cola Circular
5. Ventajas y desventajas

Colas

Es una **estructura de datos lineal** donde los elementos **se insertan en un extremo (final) y se eliminan del otro extremo (frente).**

Por esta razón, a las colas se las conoce como **FIFO (First In, First Out),** que significa "el primero en entrar es el primero en salir".



Cola en el super



Cola de impresiones



Cola de autos en el peaje

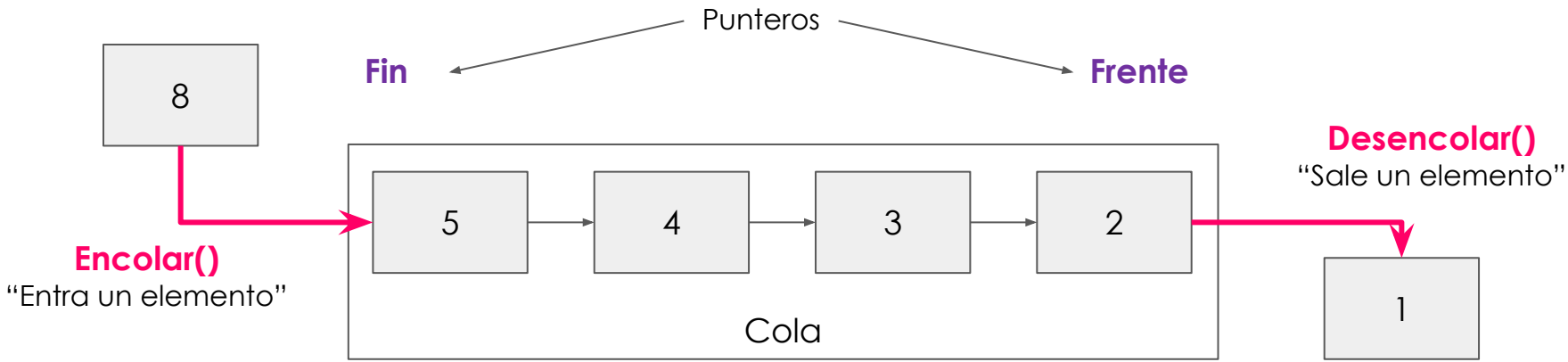
Colas

La estructura Cola está construida a partir de una **cadena de nodos conectados.**

```
typedef struct Nodo{
    int valor;          // valor que contiene el nodo
    struct Nodo* sig;   // referencia al siguiente nodo
}Nodo;
```

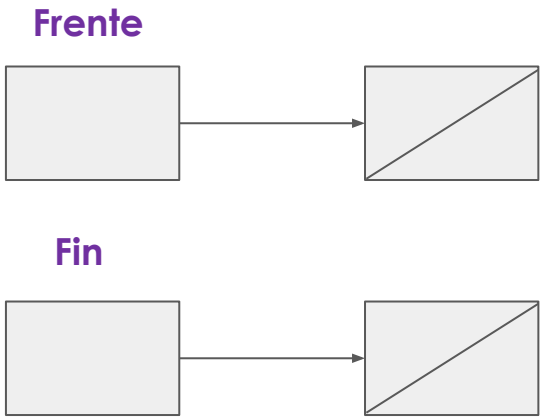
A diferencia de la Pila, requiere **dos punteros** para gestionarla eficientemente: **frente** y **fin.**

Funcionamiento



Inicializamos la cola (frente y fin)

```
...
int main() {
    Nodo* frente = NULL;
    Nodo* fin = NULL;
...
```



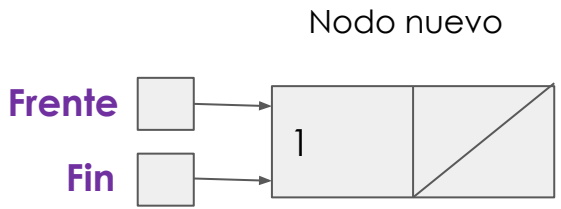
Operaciones más comunes

- Poner un nodo (encolar / enqueue): Añadir un elemento al final de la cola.
- Sacar un nodo (desencolar / dequeue): Eliminar el primer elemento de la cola.
- Verificar si la cola está vacía: Comprobar si no hay elementos.
- Mostrar todos los nodos de la cola: Recorrer la cola para ver sus elementos.
- Liberar la cola: Eliminar todos los nodos y liberar la memoria.

Función: encolar()

Esta función agrega un nodo al final de la cola. Le enviamos el **valor** "1" a guardar y los punteos **frente** y **fin** de la cola creados en el main.

```
void encolar(Nodo** frente, Nodo** fin, int v){
    // 1) Crear nodo reservando memoria
    Nodo* nuevo = (Nodo*) malloc(sizeof(Nodo));
    // 2) Asignar valores a los atributos valor y sig
    nuevo->valor = v;
    nuevo->sig = NULL;
```



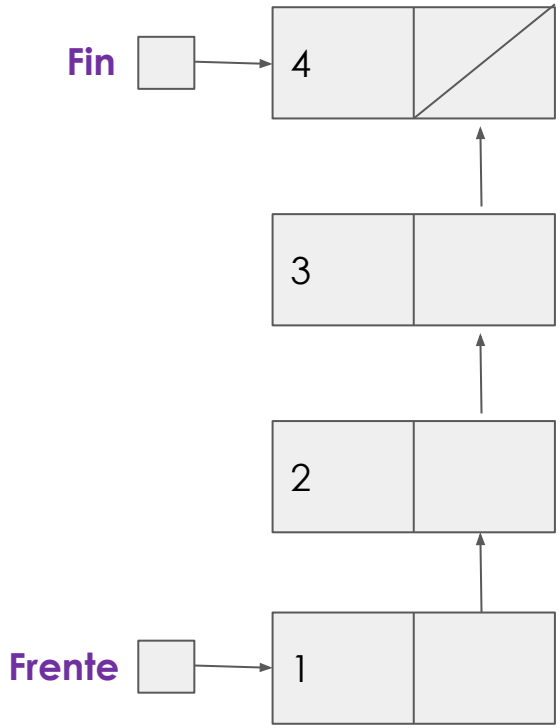
Función: encolar()

```
        // 3) Cola vacía
        if(*fin == NULL){
                *frente = nuevo; // el nuevo nodo es el frente
                *fin = nuevo;      // y también el fin

        // 4) Cola no vacía
        } else {
                (*fin)->sig = nuevo; // el último nodo apunta al nuevo
                *fin = nuevo;        // actualizamos el fin
        }
}
```



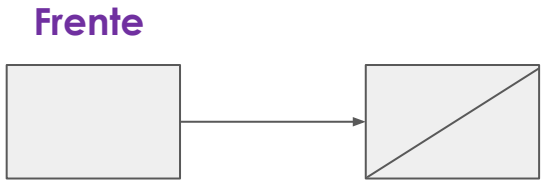
Cola de números



Función: ColaVacia()

Esta función comprueba si la cola está vacía, evaluando si el frente es NULL.

```
int colaVacia(Nodo* frente){
      return frente == NULL;
}
```



Función: desencolar()

Esta función elimina el nodo que se encuentra al frente de la cola y devuelve su valor.

```
int desencolar(Nodo** frente, Nodo** fin){

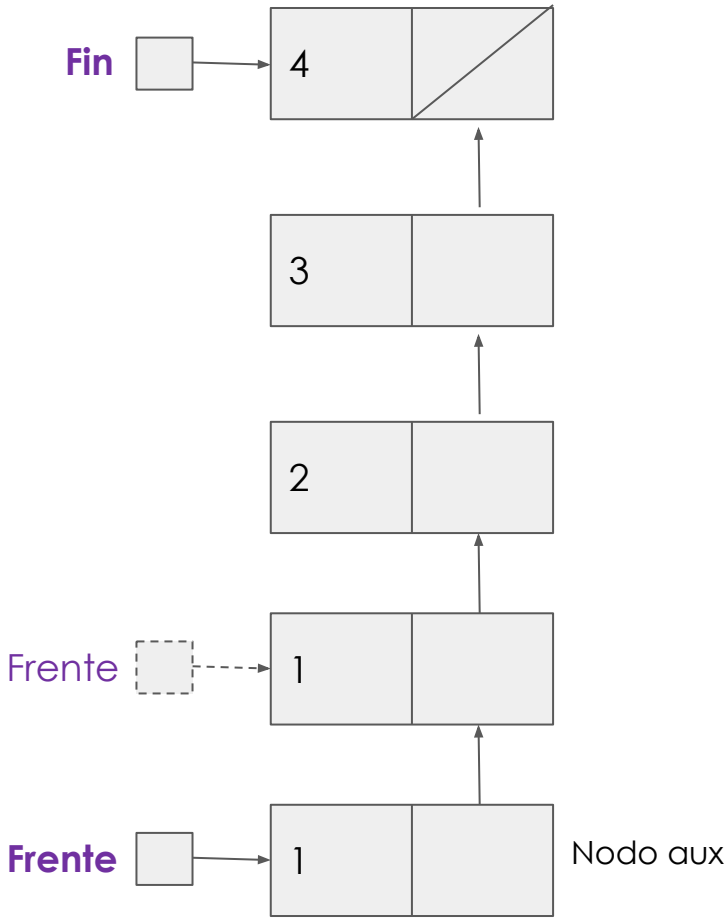
  //Comprobamos si la cola está vacía
   if(colaVacia(*frente))
          return -1;
```

Función: desencolar()

```
//Creamos un nodo auxiliar para guardar el valor del frente y luego poder liberar

// apuntamos al frente
Nodo* aux = *frente;

// guardamos el valor a devolver
int  valorDevolver= aux->valor;
```

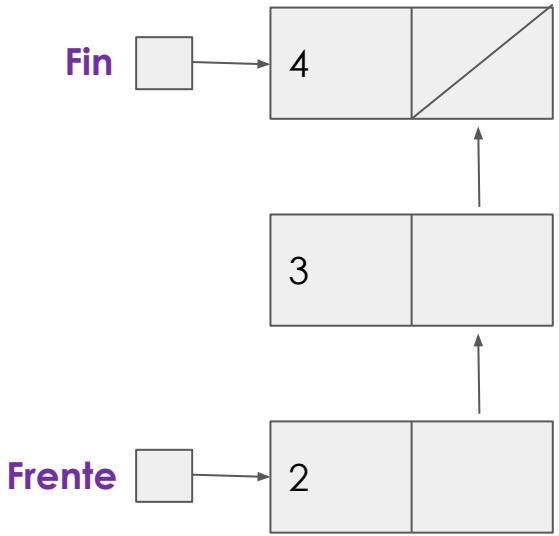


Función: desencolar()

```
// movemos el frente al siguiente nodo
*frente = aux->sig;

// Si la cola queda vacía (xq borre el último), fin es NULL
 if(*frente == NULL)
       *fin = NULL;

// liberamos memoria del nodo eliminado
    free(aux);
// devolvemos el valor
    return valorDevolver
}
```



Función: mostrar

```
void mostrar(Nodo* frente){
    // Crea un nodo auxiliar para recorrer la cola
    Nodo* aux = frente; //lo apuntamos al frente

    //Avanzamos hasta que sea NULL aux
    while(aux != NULL){
        printf("%d -> ", aux->valor);  // imprimimos el valor
        aux = aux->sig;  // avanzamos al siguiente nodo
    }
    printf("NULL\n");  // indicamos el final de la cola
}
```

Función: liberar()

```
void liberar(Nodo** frente){

    while(*frente != NULL){
        // guardamos el siguiente nodo
        Nodo* prox = (*frente)->sig;

        // liberamos la memoria del nodo actual
        free(*frente);

        // avanzamos al siguiente nodo
        *frente = prox;
    }
}
```

Llamando a las funciones: Main ()

```
int main() {

    // 1) Inicializamos la cola (frente y fin)
    Nodo* frente = NULL;
    Nodo* fin = NULL;

    // 2) Agregamos valores a la cola
    encolar(&frente, &fin, 5);
    encolar(&frente, &fin, 8);
    encolar(&frente, &fin, 7);
    encolar(&frente, &fin, 3);
    encolar(&frente, &fin, 9);

    // 3) Mostramos la cola
    printf("\nMostrar Cola\n");
    mostrar(frente);

    // 4) Sacamos el primer valor (FIFO)
    printf("\nEimina el 1er valor (FIFO): \n");
    desencolar(&frente, &fin);

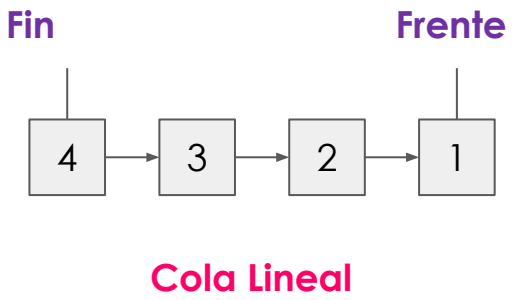
    // 5) Mostramos la cola nuevamente
    printf("\nMostrar Cola\n");
    mostrar(frente);

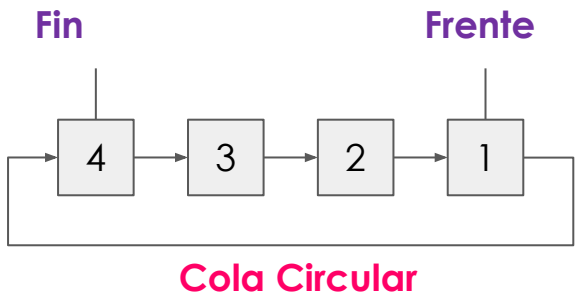
    // 6) Liberamos memoria
    printf("\nLiberando memoria...\n");
    liberar(&frente);

    return 0;
}
```

Cola Circular

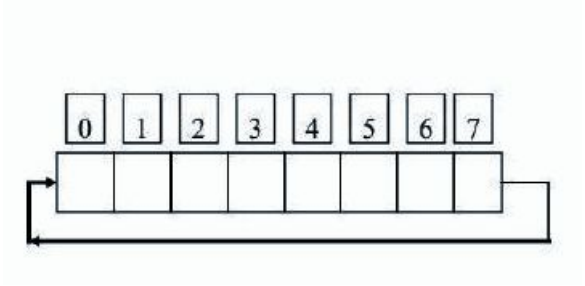
Es una variación de la cola donde el **último nodo apunta de nuevo al primero**, formando un círculo.



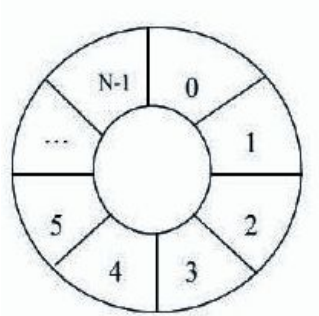


Cola Circular

Existen **dos formas** de construirla: una Cola Circular.



con arreglo (Estática)



con Nodos Enlazados (Dinámica)

Cola Circular - con arreglo (estática)

Estructura: Un arreglo de tamaño fijo y dos índices (frente, fin) que "dan la vuelta".

Memoria: Estática. No usa malloc.

Ventaja: Muy rápida y simple. Sin riesgo de fugas de memoria.

Desventaja: Tamaño fijo. Si se llena, no se pueden agregar más elementos.

Cola Circular - con nodos enlazados (dinámica)

Estructura: Nodos conectados por punteros. El último nodo apunta al primero.

Memoria: Dinámica. Usa malloc y free para cada nodo.

Ventaja: Tamaño flexible. Puede crecer según se necesite.

Desventaja: La gestión de punteros es compleja y un error puede causar bucles infinitos o fugas de memoria.

📚 Bibliografía sugerida

Sznajdleder, N. (2012). Algoritmos a fondo. Buenos Aires: Alfaomega.

Cap. 11 Estructuras de datos dinámicas lineales